

DESIGN AND ANALYSIS OF ALGORITHMS

Instructor: Dr. Mert Büyükdede



Dynamic Programming

Dynamic Programming (DP) is an optimization technique that breaks a large, complex problem into interdependent subproblems, computes each subproblem exactly once, and caches their results for reuse in subsequent computations.

Core Ideas

- **Overlapping Subproblems:** If the same subproblem would be solved multiple times, we store its result (via memoization or tabulation) to avoid redundant work.
- **Optimal Substructure:** If an optimal solution to the overall problem can be composed from optimal solutions of its subproblems, the problem exhibits optimal substructure and is flexible to DP.



Comparison

Feature	Brute-Force	Divide & Conquer	Dynamic Programming
Subproblem sharing	None	Rare	Heavy (cached)
Time complexity	Exponential	Often exponential	Typically polynomial
Typical use cases	Tiny inputs	Independent parts	Overlapping subproblems

DP Design Steps

When solving any DP problem, follow these five general stages:

1. State Definition

- Identify the parameters that define each subproblem.
- *Example (Knapsack):*
- $dp[i][j]$ = maximum value using the first i items with capacity j

2. Recurrence Relation

- Express each state in terms of smaller states.
- *Example (Knapsack):*

$$dp[i][j] = \begin{cases} dp[i-1][j], & \text{if } w_i > j, \\ \max\{ dp[i-1][j], dp[i-1][j-w_i] + v_i \}, & \text{otherwise.} \end{cases}$$

3. Base Cases

- Define the solution for the smallest subproblems.

- *Example:*

- $dp[0][*]=0, \quad dp[*][0]=0.$

4. Computation Order & Method

Top-down (Memoization): Recursively compute subproblems, storing answers in a cache (map or array).

Bottom-up (Tabulation): Iteratively fill a table (array) according to increasing subproblem size.

5. Reading the Final Answer

Extract the solution from the DP table (e.g. $dp[n][W]$) or from the memorized cache.



There are a lot of example in Dynamic Programing Approach.

- Fibonacci Sequence
- 0-1 Knapsack Problem
- Longest Common Subsequence (LCS)



Fibonacci Sequence

Let's examine the Fibonacci example step by step, illustrating both how and why Dynamic Programming (DP) improves over naïve recursion, and exploring optimizations in time and space.

Problem Statement

Compute the n th Fibonacci number, where

$$F(0)=0, \quad F(1)=1, \quad F(n)=F(n-1)+F(n-2) \quad (n \geq 2).$$

Fibonacci numbers grow exponentially if computed by the naïve recursive definition, because each call to $F(n)$ spawns two subcalls ($F(n-1)$ and $F(n-2)$), and these overlap heavily.



Bottom-up Pseudocode

```
function Fib(n):  
    dp ← array[0..n]  
    dp[0] ← 0  
    dp[1] ← 1  
    for i from 2 to n:  
        dp[i] ← dp[i-1] + dp[i-2]  
    return dp[n]
```

Top-down (Recursive with Memoization) Pseudocode

```
memo ← empty map
function FibRec(n):
    if n in memo:
        return memo[n]
    if n ≤ 1:
        return n
    memo[n] ← FibRec(n-1) + FibRec(n-2)
    return memo[n]
```

0-1 Knapsack Problem

Given items with weights w_i and values v_i , and a capacity W , the 0-1 Knapsack asks for the maximum total value of a subset whose weights sum to at most W . DP defines $dp[i][j]$ as the best value using the first i items within capacity j . Filling this table takes $O(nW)$ time, and space can be cut to $O(W)$ by using a single array in reverse. It's widely applied in resource allocation, budgeting, and cargo loading where discrete choices must be optimized under a limit.

Longest Common Subsequence (LCS)

LCS finds the longest sequence appearing (not necessarily contiguously) in both strings A and B . Define $dp[i][j]$ as the LCS length of prefixes $A[1..i]$, $B[1..j]$. If $A[i]=B[j]$, take $dp[i-1][j-1]+1$; otherwise take the max of dropping one character. The table fills in $O(mn)$ time and $O(mn)$ space. LCS is fundamental in diff/merge tools, version control, and bioinformatics for identifying shared patterns.

Coin Collecting Problem

Problem Statement:

Several coins are placed in cells of an $n \times m$ board, no more than one coin per cell.

A robot, located in the upper left cell of the board, needs to collect as many of the coins as possible and bring them to the bottom right cell.

On each step, the robot can move either one cell to the right or one cell down from its current location. When the robot visits a cell with a coin, it always picks up that coin.

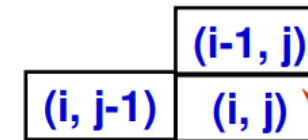
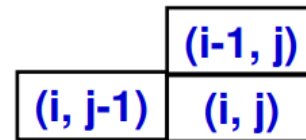
Design an algorithm to find the maximum number of coins the robot can collect and a path it needs to follow to do this.



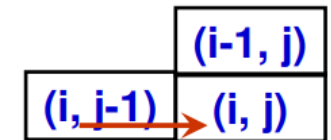
Let's create Dynamic Programming Table

	1	2	3	4	5	6
1					5	
2		4		3		
3				2		7
4			8			2
5	9				6	

- Allowed Movements

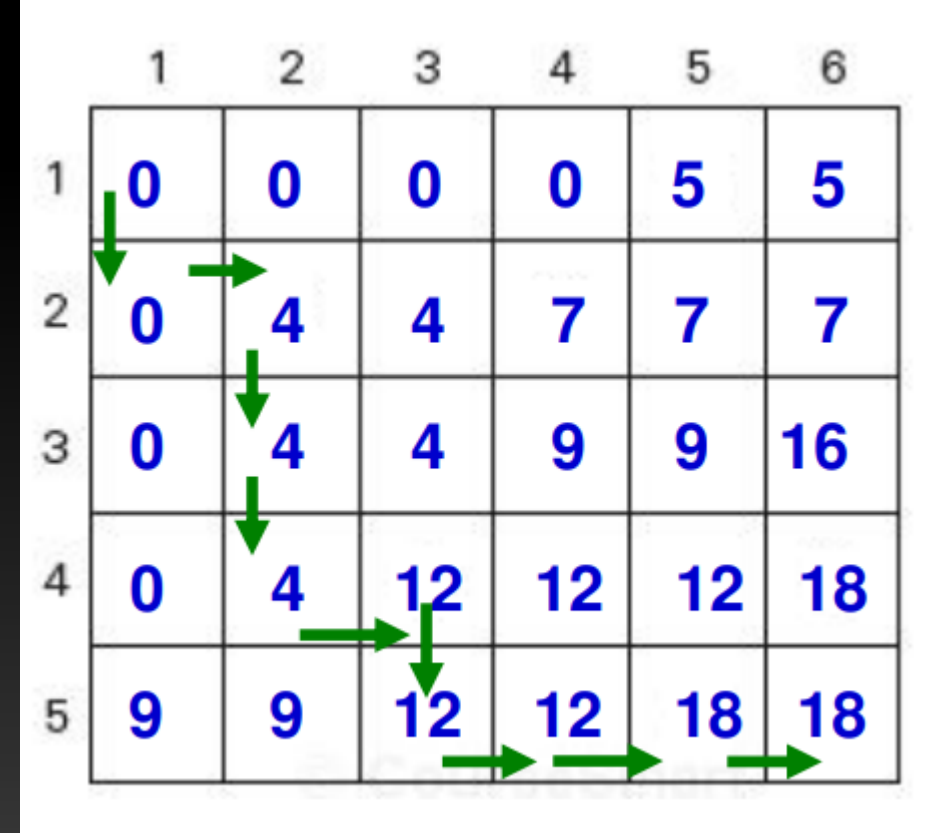
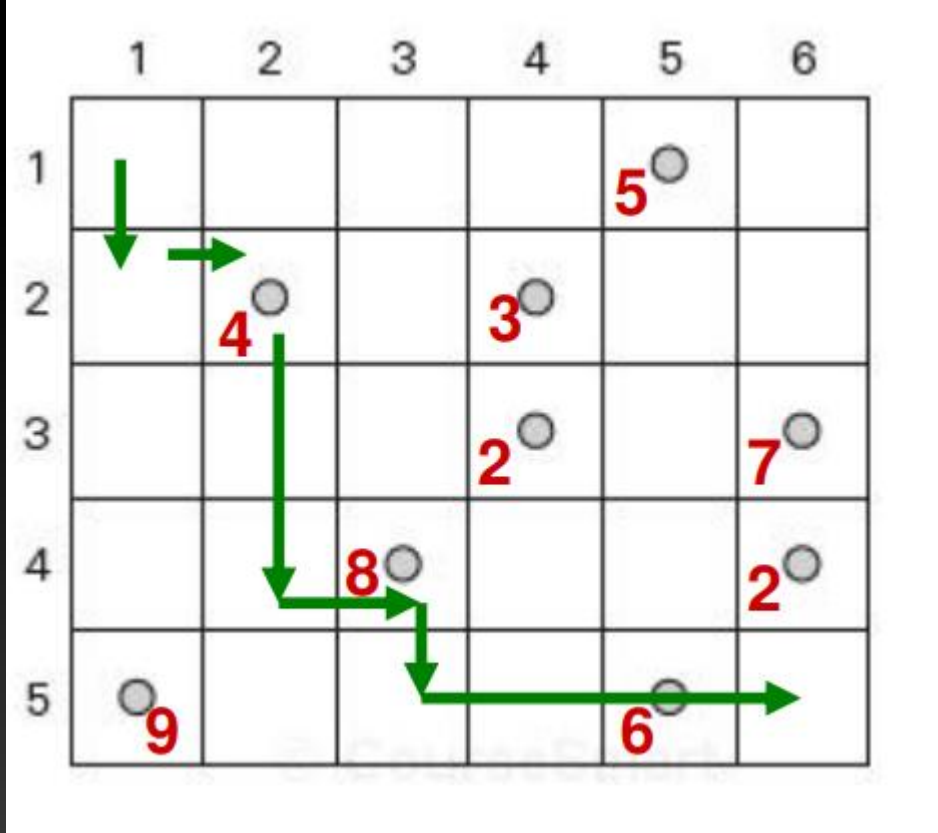


$$F(i, j) = F(i-1, j) + C_{ij}$$



$$F(i, j) = F(i, j-1) + C_{ij}$$

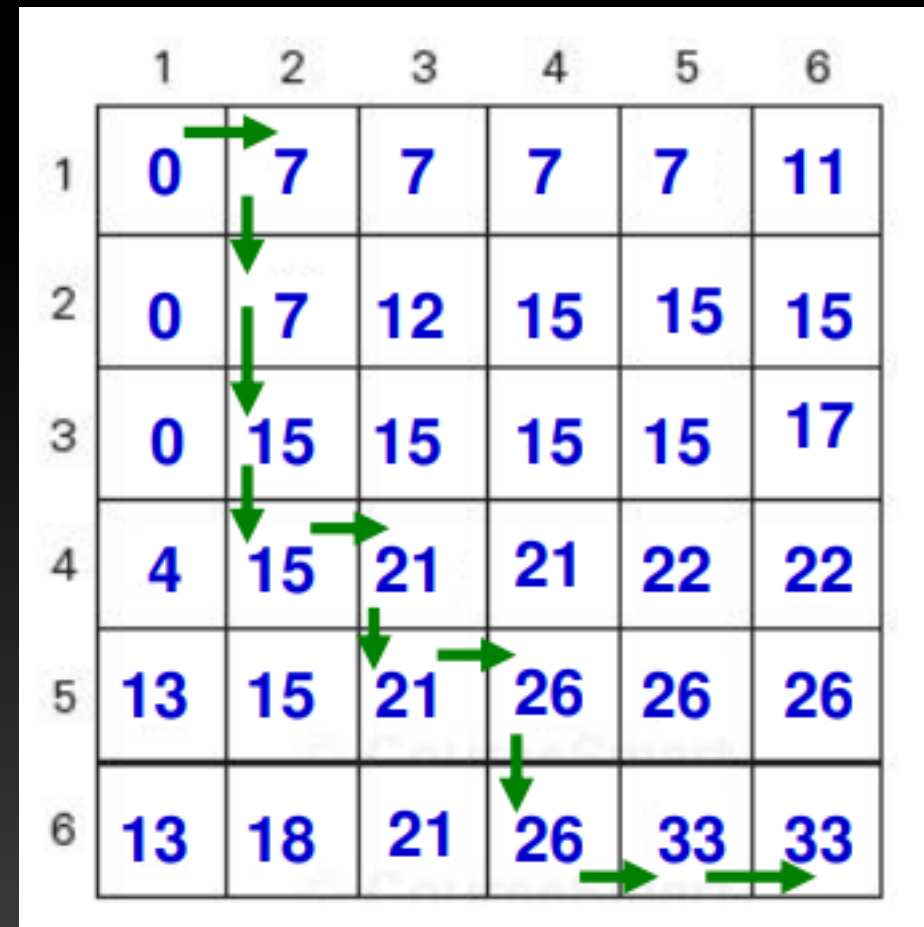
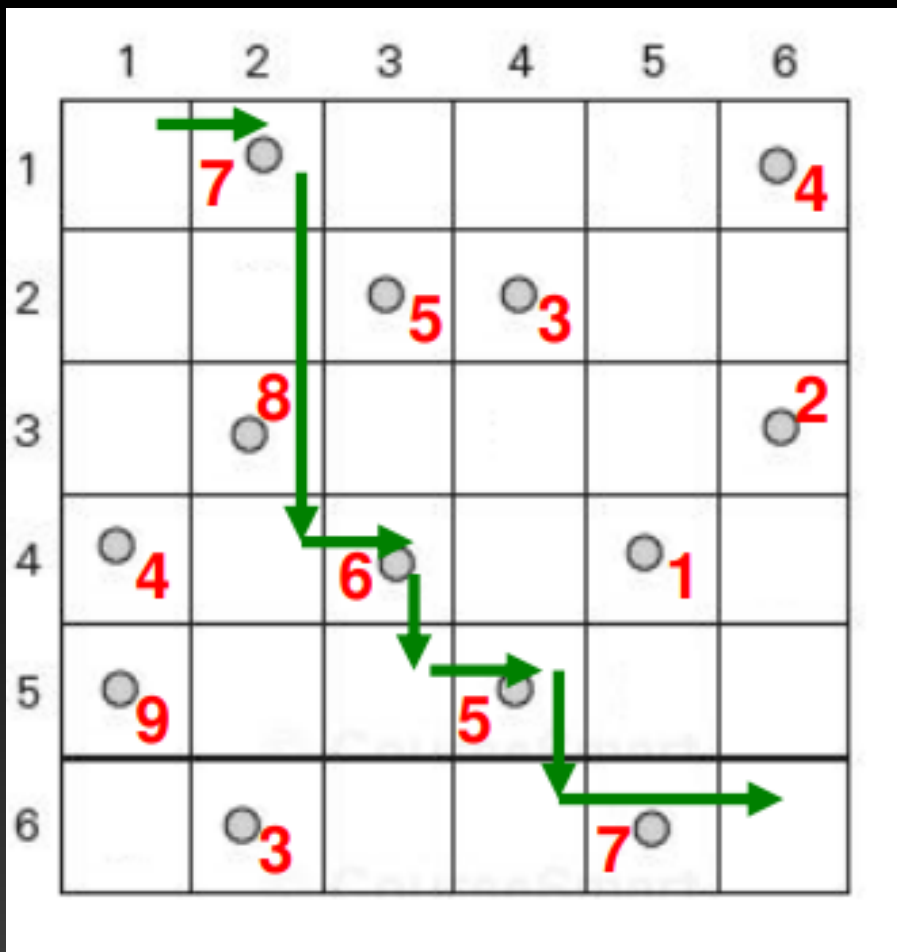
$$F(i, j) = \text{Max}\{F(i-1, j); F(i, j-1)\} + C_{ij}$$



Example

Let's create Dynamic Programming Table

	1	2	3	4	5	6
1		7				4
2			5	3		
3		8				2
4	4		6		1	
5	9			5		
6		3			7	



Example Problem

Weighted Interval Scheduling (Single-Aircraft Daily Schedule)

An airline must schedule a single aircraft for the day. You are given n flight offers; each flight j has:

- **Departure time** s_j
- **Arrival time** f_j ($f_j > s_j$)
- **Profit** p_j

Since the aircraft cannot operate more than one flight at the same time, select a subset of non-overlapping flights to **maximize total profit**.

Data Set

j	Flight ID	Departure	Arrival	Profit (USD)
1	F1	05:45	07:30	420
2	F2	06:20	08:10	380
3	F3	07:00	09:15	510
4	F4	09:10	11:20	600
5	F5	10:30	12:05	550
6	F6	12:15	14:00	480
7	F7	14:05	16:30	800

I. Compute “prev” Array

Define

$$\text{prev}[j] = \max\{i < j \mid f_i \leq s_j\},$$

or 0 if no such i exists.

j	s_j	f_j	$\text{prev}[j]$	Reason
1	05:45	07:30	0	—
2	06:20	08:10	0	Flight 1 arrives at 07:30 > 06:20 $\Rightarrow$ none
3	07:00	09:15	0	F1, F2 arrive after 07:00
4	09:10	11:20	2	F2 (08:10) $\leq$ 09:10, but F3 arrives at 09:15 > 09:10
5	10:30	12:05	3	F3 (09:15) $\leq$ 10:30
6	12:15	14:00	5	F6 (12:05) $\leq$ 12:15
7	14:05	16:30	6	F8 (14:00) $\leq$ 14:05

2. DP Definition & Recurrence

Let

$dp[j]$ = maximum total profit using flights $1 \dots j$.

Base case:

$$dp[0] = 0.$$

Recurrence for $j \geq 1$:

$$dp[j] = \max(dp[j - 1], p_j + dp[\text{prev}[j]]).$$

3. Fill the DP Table

j	prev[j]	pj	Include = $p_j + dp[prev[j]]$	Exclude = $dp[j-1]$	dp[j]
0	—	—	—	—	0
1	0	420	$420 + 0 = 420$	0	420
2	0	380	$380 + 0 = 380$	420	420
3	0	510	$510 + 0 = 510$	420	510
4	2	600	$600 + 420 = 1020$	510	1020
5	3	550	$550 + 510 = 1060$	1020	1060
6	5	480	$480 + 1060 = 1540$	1060	1540
7	6	800	$800 + 1540 = 2340$	1540	2340

4. Result & Backtracking

- **Maximum Profit:**

- $dp[7] = 2340$ USD..

- **Selected Flights:**

Backtrack from $j=7$:

- $dp[7]$ came from “include” $\Rightarrow$ select Flight 7, jump to $j = prev[7] = 6$
- $dp[6]$ came from “include” $\Rightarrow$ select Flight 6, jump to $j = 5$
- $dp[5]$ came from “include” $\Rightarrow$ select Flight 5, jump to $j = 3$
- $dp[3]$ came from “include” $\Rightarrow$ select Flight 3, jump to $j = 0$

- **Chosen:** Flights 3, 5, 6, 7

Total Profit: $510 + 550 + 480 + 800 = 2340$ USD.

Let's try to write Algorithm with PseudoCode for DP.

```
Input:
    flights[1..n] // each flight has start, finish, profit

// 1. Sort flights by finish time
sort flights by flights[i].finish increasing

// 2. Compute prev[1..n]
for j from 1 to n:
    prev[j] ← 0
    for i from j-1 down to 1:
        if flights[i].finish ≤ flights[j].start:
            prev[j] ← i
            break

// 3. Build DP table
dp[0] ← 0
for j from 1 to n:
    include ← flights[j].profit + dp[ prev[j] ]
    exclude ← dp[j-1]
    dp[j] ← max(include, exclude)

// 4. Backtrack to find selected flights
selected ← empty list
j ← n
while j > 0:
    if flights[j].profit + dp[ prev[j] ] > dp[j-1]:
        prepend flights[j] to selected
        j ← prev[j]
    else:
        j ← j-1

// Output
return dp[n], selected
```

Activity:

Below you will see a new DP scenario called “Film Festival Theater Planning”. You have a single theater and movies of different lengths will be shown throughout the day. Each movie has a start-end time and an expected number of viewers. Your goal is to maximize the total number of viewers by selecting movies that do not overlap.

j	Film Title	Start – End	Expected Viewers
1	“Dawn of Dreams”	09:00–10:30	200
2	“Midday Melody”	10:00–11:15	150
3	“Noon Noir”	10:45–12:30	300
4	“Afternoon Aura”	12:00–13:30	250
5	“Sunset Sonata”	13:00–14:00	100
6	“Evening Epic”	13:30–15:00	180
7	“Night Narrative”	14:30–16:00	220

The festival runs from 09:00 to 16:00 in a single screening room.



Task 1: Computing prev[j]

For each film j , find the largest index $i < j$ whose end time “does not overlap” with film j ’s start:

j	End	prev[j]	Explanation
1	10:30	0	—
2	11:15	0	Film 1 ends 10:30 > 10:00 (overlaps)
3	12:30	1	Film 1 ends 10:30 ≤ 10:45
4	13:30	2	Film 2 ends 11:15 ≤ 12:00
5	14:00	3	Film 3 ends 12:30 ≤ 13:00
6	15:00	4	Film 4 ends 13:30 ≤ 13:30
7	16:00	5	Film 5 ends 14:00 ≤ 14:30

Task: 2 DP Definition & Recurrence

State:

$dp[j] = \text{max total viewers from films } 1 \dots j \text{ with no overlaps.}$

Base Case:

$$dp[0] = 0.$$

Recurrence:

$$dp[j] = \max(dp[j - 1], \text{viewers}_j + dp[\text{prev}[j]]).$$

- **Exclude:** skip film j , keep $dp[j - 1]$.
- **Include:** add viewers of j plus best up to `prev[j]`.

Task 3: Step-by-Step Calculation

j	prev[j]	viewers_j	Include = viewers_j + dp[prev[j]]	Exclude = dp[j-1]	dp[j]
0	—	—	—	—	0
1	0	200	$200 + 0 = 200$	0	200
2	0	150	$150 + 0 = 150$	200	200
3	1	300	$300 + 200 = 500$	200	500
4	2	250	$250 + 200 = 450$	500	500
5	3	100	$100 + 500 = 600$	500	600
6	4	180	$180 + 500 = 680$	600	680
7	5	220	$220 + 600 = 820$	680	820

Task 4: Pseudocode

```
// Film Festival Scheduling (Weighted Interval Scheduling)
```

```
Input:
```

```
  n = 7
```

```
  start[1..n], end[1..n], viewers[1..n]
```

```
// 1. Compute prev[1..n]
```

```
for j = 1 to n:
```

```
  prev[j] = 0
```

```
  for i = j-1 down to 1:
```

```
    if end[i] ≤ start[j]:
```

```
      prev[j] = i
```

```
      break
```

```
// 2. DP array
```

```
dp[0] = 0
```

```
for j = 1 to n:
```

```
  include = viewers[j] + dp[ prev[j] ]
```

```
  exclude = dp[j-1]
```

```
  dp[j] = max(include, exclude)
```

Task 4: Pseudocode

```
// 3. Backtracking to recover selection
selected = empty list
j = n
while j > 0:
    if viewers[j] + dp[ prev[j] ] > dp[j-1]:
        prepend j to selected
        j = prev[j]
    else:
        j = j-1

Output dp[n], selected
```

Task 5: Java Implementation

```
import java.util.*;

public class FilmFestival {
    public static void main(String[] args) {
        int n = 7;
        String[] films = {
            "",
            "Dawn of Dreams", "Midday Melody", "Noon Noir",
            "Afternoon Aura", "Sunset Sonata", "Evening Epic", "Night Narrative"
        };
        int[] start = {0, 900, 1000, 1045, 1200, 1300, 1330, 1430};
        int[] end = {0, 1030, 1115, 1230, 1330, 1400, 1500, 1600};
        int[] viewers = {0, 200, 150, 300, 250, 100, 180, 220};

        // 1. Compute prev[]
        int[] prev = new int[n+1];
        for (int j = 1; j <= n; j++) {
            prev[j] = 0;
            for (int i = j-1; i >= 1; i--) {
                if (end[i] <= start[j]) {
                    prev[j] = i;
                    break;
                }
            }
        }
    }
}
```

Task 5: Java Implementation

```
// 2. Fill dp[]
int[] dp = new int[n+1];
dp[0] = 0;
for (int j = 1; j <= n; j++) {
    int include = viewers[j] + dp[ prev[j] ];
    int exclude = dp[j-1];
    dp[j] = Math.max(include, exclude);
}

// 3. Output result and selection
System.out.println("Max Total Viewers: " + dp[n]);
List<String> selected = new ArrayList<>();
int j = n;
while (j > 0) {
    if (viewers[j] + dp[ prev[j] ] > dp[j-1]) {
        selected.add(films[j]);
        j = prev[j];
    } else {
        j--;
    }
}
Collections.reverse(selected);
System.out.println("Selected Films: " + selected);
}
```